PRD: EcoStruxure Edge Load Balancer

Nov 7, 2026 · @AnyaAgidi

Summary

- **Problem:** Commercial buildings pay heavy demand charges when they draw grid power during peak price hours, and shifting load by hand is too slow.
- **For whom:** Commercial facility operations managers with on-site storage.
- **What was built:** A simulated edge engine that reads the price and battery level every cycle, chooses grid or internal power, and splits the load across battery, solar and generator when it goes internal. An operator page shows the source, battery level, money saved and percent saved.
- **Result:** In one simulated day the system saved 24.6% ($43.00 of $175.00) against a 15 to 20% target. The battery never went below 75% against a 30% floor, and each decision took far less than the 500 ms target.
- ***Caveat:** The result comes from simulated prices and source sizes, so it shows the logic works, not what a real site would save.

1. Product Overview and Problem Statement

EcoStruxure Edge Load Balancer switches a building's power draw from the grid to backup batteries whenever energy prices spike.

- **Problem:** Commercial facility managers pay heavy demand charges when they draw power during peak grid hours. Shifting load by hand is too slow, so utility bills stay high.
- **Solution:** An automated edge-logic engine reads live tariff rates and battery state of charge, then moves the building onto battery power during price spikes and back to the grid when prices drop.
- **Purpose of this build:** A working proof of concept that shows automated, tariff-based load shifting can cut peak demand charges, built to run on simulated data with no hardware dependency.

2. Target Persona and Value Proposition

The user is a Commercial Facility Operations Manager who is judged on uptime and on the utility bill.

- **Persona:** Commercial Facility Operations Manager at an office, retail or light industrial site with on-site battery storage.
- **Pain:** Peak-hour demand charges are the largest avoidable line on the bill, and nobody is watching the tariff clock all day.
- **Value:** Cuts peak energy costs by 15 to 20% automatically, with no interruption to power.
- **Why edge:** The decision runs locally, so it keeps working if the cloud connection drops.

3. MVP Scope

The MVP has three must-haves; anything that does not prove the cost-saving story is cut.

In scope (must have)

- Data ingestion for simulated tariff prices ($/kWh) and battery state of charge (% SoC).
- Rule-based decision engine: **IF** tariff > threshold and battery > 30% **THEN** switch to battery.
- Minimal operator UI showing the live power source and money saved.

Out of scope (cut for demo)

- Physical hardware wiring.
- AI-based predictive price forecasting.
- Multi-facility enterprise management.

4. Success Metrics (KPIs)

The demo succeeds if it shows a 15 to 20% peak-hour saving and decides in under 500 ms.

Type	Metric	Target	How it is measured
Business	Peak-hour cost reduction	15 to 20%	Simulated bill with the engine on vs. grid-only, same load and tariff profile
Product	Edge decision latency	Under 500 ms	Time from new tariff or SoC reading to the switch decision, logged per cycle
Safety	Battery floor respected	SoC never below 30%	Check in the decision log after a full simulated day

5. Functional Requirements

Each requirement is testable in the demo.

ID	Requirement	Acceptance criteria
FR-1	Ingest simulated tariff price ($/kWh) on a fixed interval	New price is read and timestamped every cycle; a spike scenario can be triggered on demand
FR-2	Ingest simulated battery SoC (%)	SoC drops while on battery and recovers while on grid, within the simulation
FR-3	Evaluate the switching rule each cycle	Battery is selected when tariff is above the threshold and SoC is above 30%; grid otherwise
FR-4	Configurable tariff threshold	Operator can change the threshold and see the next decision reflect it

ID	Requirement	Acceptance criteria
FR-5	Show live power source	UI displays Grid or Battery and updates on every decision
FR-6	Calculate money saved	UI shows running savings vs. a grid-only baseline over the same period
FR-7	Log every decision	Each entry holds time, tariff, SoC, source chosen and decision latency

6. Technical Approach with timed plan

A small Python service runs the loop and a lightweight web page shows the result and the stack below is a suggestion to adjust.

1. Tariff simulator and battery simulator produce readings each cycle.
2. Decision engine applies the rule and picks Grid or Battery.
3. Savings calculator compares the choice to a grid-only baseline.
4. Operator UI shows the source, SoC, tariff and running savings.

Window	Goal	Done when
Day 0	Simulators and data model	Tariff and SoC streams run with a spike scenario
Day 0	Decision engine and logging	Rule passes tests at the 30% SoC boundary; latency logged
Day 1	Operator UI and savings calculation	Live source and money saved update on screen
Day 2	Test a full simulated day and tune the threshold	Savings land in the 15 to 20% range
Day 2	README, short demo recording, publish	A recruiter can watch the demo and read the repo in 5 minutes

Risks: The 15 to 20% saving depends on the simulated tariff curve and load profile, so document those assumptions in the README. Scope creep into forecasting or hardware is the main schedule risk.

7. Milestones and Tasks

Five milestones follow the 2-day windows in section 6.

M1: Simulators running (Day 0)

- ☐ Set up the repo, folder structure and README skeleton
- ☐ Write the tariff simulator with a daily price curve ($/kWh)
- ☐ Add a spike scenario that can be triggered on demand

- ☐ Write the battery simulator (% SoC drains on battery, recovers on grid)
- ☐ Define the shared reading format (timestamp, tariff, SoC)

M2: Decision engine working (Day 0)

- ☐ Implement the switching rule with a configurable tariff threshold
- ☐ Enforce the 30% SoC floor
- ☐ Write tests for the boundaries (SoC at 29, 30 and 31%; tariff at the threshold)
- ☐ Log every decision with time, tariff, SoC, source and latency
- ☐ Measure decision latency against the 500 ms target

M3: Operator UI live (Day 1)

- ☐ Build the page showing current source (Grid or Battery), SoC and tariff
- ☐ Add the threshold control
- ☐ Build the grid-only baseline calculation
- ☐ Show running money saved on screen

M4: Validated against KPIs (Day 2)

- ☐ Run a full simulated day with the engine on and off
- ☐ Compare the two bills and check the 15 to 20% saving
- ☐ Tune the threshold and the load and tariff profile if the result is off
- ☐ Confirm SoC never dropped below 30% in the log

M5: Packaged (Day 2)

- ☐ Write the README with the problem, approach, assumptions and results
- ☐ Record a short demo of a price spike and the switch
- ☐ Push to GitHub and check it runs from a clean clone

8. *Build Update and Results

The original targets in section 4 are unchanged. This section records what was built and how it was measured against them.

Changes from the original scope

- The engine now decides between grid and internal power. When it picks internal, a load balancer splits the building's load across battery, solar and generator in proportion to the energy each has available.
- Added a price threshold control on the operator page, so the switching price can be changed live.
- Added a price spike checkbox. This is a demo control only. In a real deployment the price comes from a live utility feed and no manual step exists.
- Added a percent saved readout, a decision log (decision_log.csv) with per-decision latency, and automated tests for the decision engine boundaries.
- Battery recharge cost from the grid is now counted in the cost with the system.

Simulation assumptions

- Tariff: 0.08 $/kWh overnight, 0.15 daytime, 0.30 from 5 PM to 9 PM.
- Building load: constant 50 kW. Battery: 400 kWh, recharged from the grid at 25 kW.
- Internal sources: battery 200 kWh at 0.05 $/kWh, solar 100 kWh at 0.00, generator 100 kWh at 0.12.

Results (one simulated day, default settings)

Metric	Target	Result
Peak-hour cost reduction	15 to 20%	24.6% ($43.00 saved on a $175.00 grid-only bill)
Edge decision latency	Under 500 ms	Far under target (well below 1 ms per decision)
Battery floor respected	SoC never below 30%	Lowest level 75%

The saving is a simulation result under the assumptions above, not a prediction for a real site.

9. Decisions and Tradeoffs

Decision	Why	Tradeoff
Cut hardware and SCADA wiring	The core value is the switching logic, and hardware would not fit a 2 day build	Nothing is tested against a real device or meter
Cut AI price forecasting	A fixed threshold rule is simple to explain and test	The engine reacts to prices instead of anticipating them
Cut multi-facility management	One building is enough to prove the idea	No view across sites
Kept the 15 to 20% target and reported the actual 24.6%	Changing a target after seeing the result would make it meaningless	The result needs its assumptions listed beside it
Split the load by available energy	It is simple and every source contributes	The split ignores cost, so it is not the cheapest possible mix
Used simulated price data and a demo-only spike checkbox	No live feed was available	In a real deployment only the price function would be replaced and no manual step would remain

10. Next Steps and Roadmap

The order below puts the work that makes the result believable before the work that makes the product bigger.

1. **Real price feed:** Replace the simulated tariff function with a live utility or smart meter feed. This is the smallest change and turns the demo into something that reacts to real prices.
2. **Cost-aware load split:** Choose how much each source carries by cost as well as available energy, so the mix is closer to the cheapest one.
3. **Validate with a facility manager:** Test the assumptions and the savings estimate against a real building's load and tariff.
4. **Price forecasting:** Predict price spikes so the battery is ready before they arrive.
5. **Hardware integration:** Connect to real meters and controllers.
6. **Multi-facility management:** Show and control several buildings from one place.

11. Risks and Open Questions

- The savings figure depends on simulated prices, load and source sizes, so it may not hold at a real site.
- It is unknown whether real facility managers would trust an automatic switch without an override.
- Generator fuel cost, battery wear and local utility rules are not modeled.

12. Agile Approach

The project was built as a single 2-day time-boxed sprint by one person, using agile practices.

- MVP first: The scope was cut to the smallest version that proves the value, with out-of-scope items written down so they could not creep back in.
- Prioritized backlog: The functional requirements (FR-1 to FR-7) and the roadmap in section 10 act as the backlog, ordered by how much each item makes the result believable.
- Small increments: Work was split into five milestones (M1 to M5). Each one ended with something that runs: simulators, then the decision engine, then the operator page, then validation, then packaging.
- Working software at every step: The code was run after every change and saved in small commits, so there was always a version that worked.
- Tests on the riskiest logic: Boundary tests cover the 30% battery floor and the price threshold, since a wrong decision there is the costliest failure.
- Inspect and adapt: The plan changed when the build showed a reason to. The load balancer and the solar and generator sources were added to the original battery-only scope, and the recharge cost was added after the first savings figure turned out to be overstated.
- Definition of done: Each milestone was finished against measurable targets (percent saved, decision latency, battery floor) instead of a feeling that it was done.
- Not yet done: A normal agile loop includes user feedback each iteration. This build has none yet, which is why validating with a facility manager is the third item on the roadmap.